

PRESIDENCY UNIVERSITY

CSE211: Data Structures and Algorithms

Lecture 04: Stacks and Queues

Understanding LIFO, FIFO, representations, and applications

Afzal Hossan

Lecturer

Department of Computer Science and Engineering

Presidency University

Learning goals: explain, trace, and choose

Understand

- Define a **stack** and a **queue** as ADTs.
- Explain LIFO vs FIFO using diagrams.
- Trace how elements move and identify boundary cases.

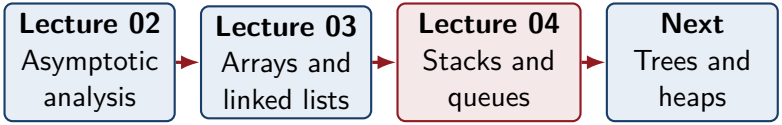
Apply

- Compare arrays, linked lists, and circular arrays.
- Reason about operation time and storage.
- Solve brackets, expression, and waiting-line examples.

Key idea

Focus on **what changes after each operation** and **why the ordering rule holds**.

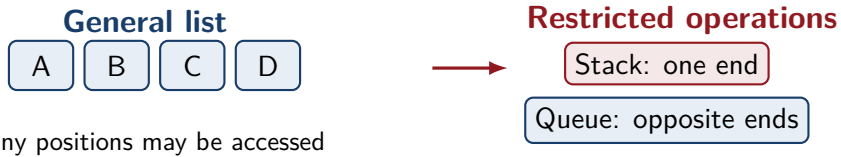
Where Lecture 04 fits in the course



Connecting idea

An array or linked list tells us *where data lives*. An ADT tells us *which operations users are permitted to perform*.

Why invent another structure when lists already exist?



Question for the class

When you undo your last edit, should the *oldest* or the *most recent* action be reversed?

Why invent another structure when lists already exist?



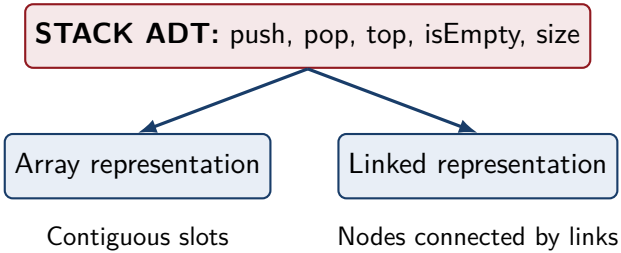
Question for the class

When you undo your last edit, should the *oldest* or the *most recent* action be reversed?

Answer

The most recent action: that ordering rule is exactly LIFO.

An ADT is a contract, not a memory diagram

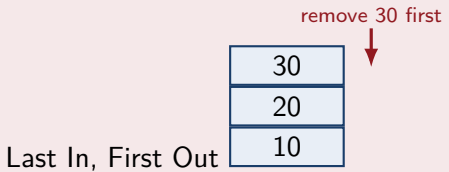


Key idea

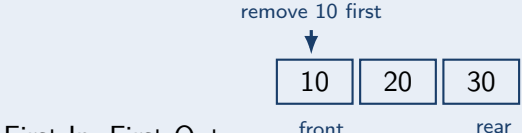
The **ADT fixes behavior**. The **implementation** determines memory organization and performance details.

Two ordering rules: LIFO and FIFO

LIFO: stack



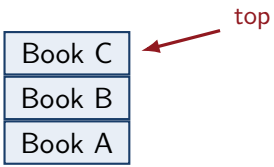
FIFO: queue



Same arrivals: 10, 20, 30

Different departures: stack 30, queue 10

Stack vocabulary: the only accessible end is top



- **Push:** put a book on top.
- **Pop:** take the top book away.
- **Peek / top:** read the top book without removing it.

Key idea

A stack never allows direct deletion of a buried element through its standard ADT.

Stack operations: what can change?

Operation	What it means	Changes stack?
push(x)	Insert x at top	Yes
pop()	Return and remove top element	Yes
top()/peek()	Return top without removal	No
isEmpty()	Is there no element?	No
size()	How many elements?	No

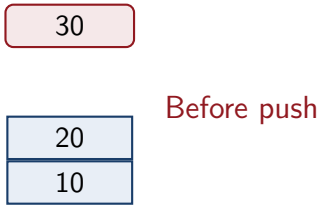
Stack operations: what can change?

Operation	What it means	Changes stack?
push(x)	Insert x at top	Yes
pop()	Return and remove top element	Yes
top()/peek()	Return top without removal	No
isEmpty()	Is there no element?	No
size()	How many elements?	No

Mini-check

After `top()` returns 40, would a subsequent `pop()` return 40 or the element below it? **It returns 40: peek did not change the stack.**

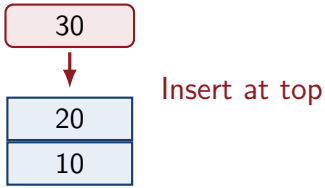
Animated trace: push 30 on the stack



Invariant

New element becomes the top; previously stored elements keep their relative order.

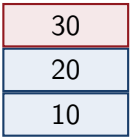
Animated trace: push 30 on the stack



Invariant

New element becomes the top; previously stored elements keep their relative order.

Animated trace: push 30 on the stack



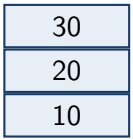
After push

top = 30

Invariant

New element becomes the top; previously stored elements keep their relative order.

Animated trace: pop removes the top element

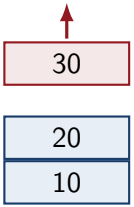


Before pop

Invariant

Pop changes the stack; a later top() now observes the element below the old top.

Animated trace: pop removes the top element

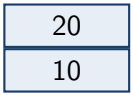


Return 30

Invariant

Pop changes the stack; a later top() now observes the element below the old top.

Animated trace: pop removes the top element



After pop

new top = 20

Invariant

Pop changes the stack; a later top() now observes the element below the old top.

Stack challenge: determine every output

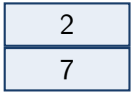
1. push(7), push(2), push(9)
2. top() = **?**
3. pop() = **?**
4. push(4)
5. pop() = **?**
6. top() = **?**

What is the final stack?

Stack challenge: determine every output

1. push(7), push(2), push(9)
2. top() = **9**
3. pop() = **9**
4. push(4)
5. pop() = **4**
6. top() = **2**

Final stack:
`[7, 2]`



Underflow and Overflow: What Can Go Wrong?

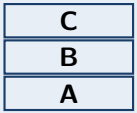
Underflow: Empty Stack



size = 0

No element exists to **remove or inspect**.

Overflow: Full Stack

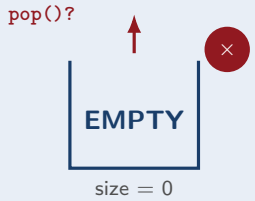


size = capacity = 3

No free position for **another element**.

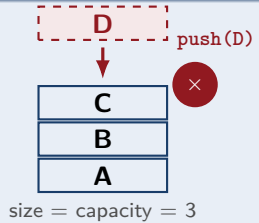
Underflow and Overflow: What Can Go Wrong?

Underflow: Empty Stack



No element exists to **remove or inspect**.

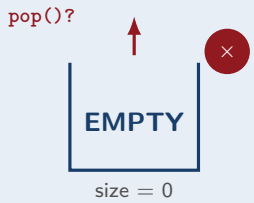
Overflow: Full Stack



No free position for **another element**.

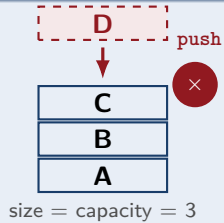
Underflow and Overflow: What Can Go Wrong?

Underflow: Empty Stack



No element exists to **remove or inspect**.

Overflow: Full Stack



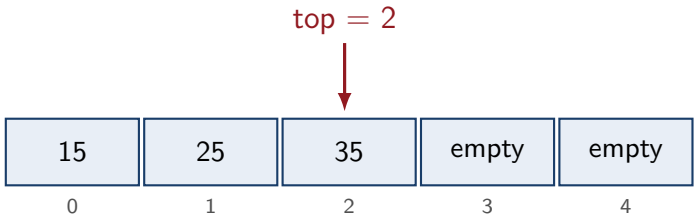
No free position for **another element**.

Underflow: an invalid operation on an empty stack.

Key idea

Overflow: exceeding fixed capacity.
A **dynamic** stack can grow when full, but it must still handle underflow.

Array representation: keep track of top



Initial state

Empty stack: $top = -1$. First push stores at index 0.

Updates

Push increments top; pop reads the current slot and decrements top.

Why can push be slow once in a while?

Array is full

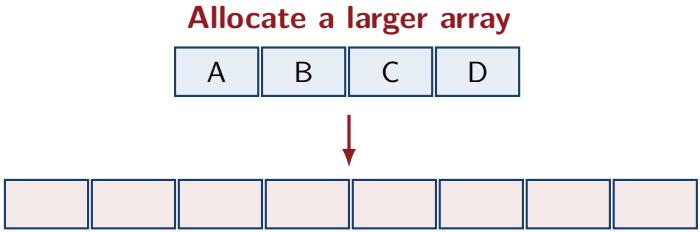


capacity 4; size 4

Time complexity insight

Ordinary push is $O(1)$; growth may copy n items and cost $O(n)$. Under geometric expansion, many pushes average $O(1)$ **amortized**.

Why can push be slow once in a while?



Time complexity insight

Ordinary push is $O(1)$; growth may copy n items and cost $O(n)$. Under geometric expansion, many pushes average $O(1)$ **amortized**.

Why can push be slow once in a while?

Copy old entries; then append E

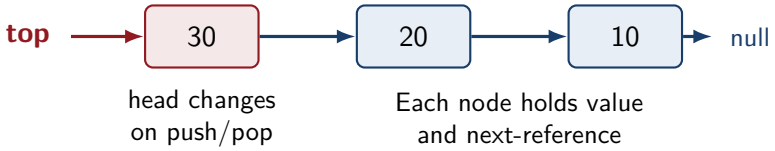


capacity 8; size 5

Time complexity insight

Ordinary push is $O(1)$; growth may copy n items and cost $O(n)$. Under geometric expansion, many pushes average $O(1)$ **amortized**.

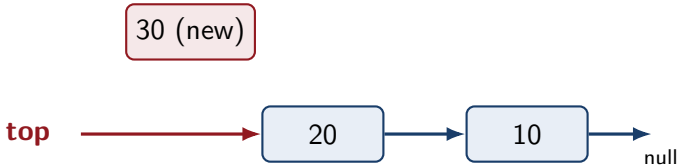
Linked stack: the head node is the top



Why keep top at the head?

Inserting or removing the first linked node changes only a constant number of references: $O(1)$.

Linked-stack push: update the references

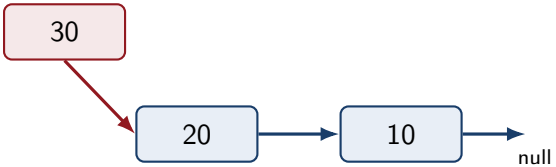


Create node 30

Correctness

Link the new node to the old top *before* changing the top reference. No earlier node is lost.

Linked-stack push: update the references



Point 30 to old top

Correctness

Link the new node to the old top *before* changing the top reference. No earlier node is lost.

Linked-stack push: update the references



Move top to new node

Correctness

Link the new node to the old top *before* changing the top reference. No earlier node is lost.

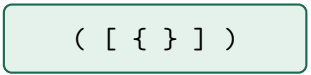
Stack representations: which one should we choose?

Aspect	Dynamic array	Singly linked list
Top access	$O(1)$	$O(1)$
Push	$O(1)$ amortized	$O(1)$
Pop	$O(1)$	$O(1)$
Storage	Contiguous; spare capacity	One node/link per element
Strength	Cache-friendly, compact	No large relocation

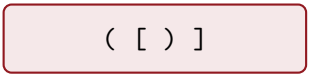
Big idea

The **LIFO rule never changes**. Only the representation, memory overhead, and occasional resizing cost change.

Application 1: why brackets are a stack problem

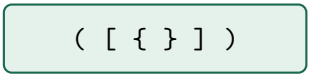


Properly nested

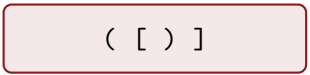


Crossed closing order

Application 1: why brackets are a stack problem



Properly nested



Crossed closing order

Rule

On an opening bracket, push it. On a closing bracket, it must match the **most recent unmatched opening bracket**.

Why not count brackets?

`([)]` has the same total counts of opening and closing brackets, yet its nesting is invalid.

Animated bracket matcher: ([{ }])

([{ }])



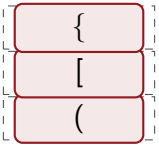
Start empty

Acceptance condition

No mismatch occurs *and* the stack is empty after scanning the entire input.

Animated bracket matcher: ([{ }])

([{ }])



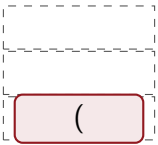
Push three openings

Acceptance condition

No mismatch occurs *and* the stack is empty after scanning the entire input.

Animated bracket matcher: ([{ }])

([{ }])



Close } then]
Pop matching openings

Acceptance condition

No mismatch occurs *and* the stack is empty after scanning the entire input.

Animated bracket matcher: ([{ }])

([{ }])



Close)

Acceptance condition

No mismatch occurs *and* the stack is empty after scanning the entire input.

Bracket matching: three distinct failure cases

Input	What goes wrong	Reason
`([)]`	**Wrong match**	Closing) sees top [.
`())`	**Premature close**	A closing bracket arrives after stack becomes empty.
`(((`	**Unclosed opening**	Stack remains nonempty at end.

Complexity

For n characters, each character is examined once: $O(n)$ time, $O(n)$ worst-case auxiliary space for unmatched openings.

Application 2: why use prefix and postfix?

Infix: `( 6 + 2 ) * 3`

Prefix: `* + 6 2 3`

Postfix: `6 2 + 3 *`

Key idea

Postfix removes the need for parentheses or precedence rules *during evaluation*: a stack stores intermediate operands.

Postfix evaluation: see every operation

Token	What happens	Stack (bottom $\rightarrow$ top)
6	Push 6	[6]
2	Push 2	[6, 2]
3	Push 3	[6, 2, 3]
+	Pop 3 and 2; push $2 + 3 = 5$	[6, 5]
*	Pop 5 and 6; push $6 \cdot 5 = 30$	[30]
4	Push 4	[30, 4]
-	Pop 4 and 30; push $30 - 4 = 26$	[26]

Postfix evaluation: see every operation

Token	What happens	Stack (bottom $\rightarrow$ top)
6	Push 6	[6]
2	Push 2	[6, 2]
3	Push 3	[6, 2, 3]
+	Pop 3 and 2; push $2 + 3 = 5$	[6, 5]
*	Pop 5 and 6; push $6 \cdot 5 = 30$	[30]
4	Push 4	[30, 4]
-	Pop 4 and 30; push $30 - 4 = 26$	[26]

Answer

6 2 3 + * 4 - $\Longrightarrow$ **26**

Operand order is crucial for subtraction and division



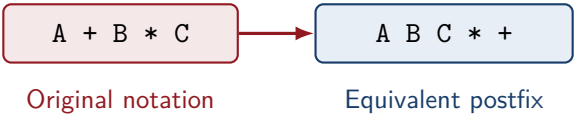
Encountering –

First popped: $b = 5$ (right operand).
Second popped: $a = 20$ (left operand).
Compute $a - b = 20 - 5 = 15$.

Common error

Computing $5 - 20$ reverses the operands. The same issue appears with division.

Converting infix to postfix: the intuition



1. Output operands as they appear.
2. Temporarily keep operators on a stack.
3. Pop higher-priority operators before lower-priority ones.
4. Parentheses determine when an operator group is complete.

Key idea

$\times$ and $\div$ outrank $+$ and $-$. Equal precedence requires an associativity rule.

Animated infix-to-postfix: `A + B * C`

Read	Action	Operator stack	Output
A	Output operand	[ ]	A
+	Push operator	[+]	A
B	Output operand	[+]	A B
*	Higher precedence: push	[+, *]	A B
C	Output operand	[+, *]	A B C
end	Pop * then +	[ ]	A B C * +

Animated infix-to-postfix: `A + B * C`

Read	Action	Operator stack	Output
A	Output operand	[ ]	A
+	Push operator	[+]	A
B	Output operand	[+]	A B
*	Higher precedence: push	[+, *]	A B
C	Output operand	[+, *]	A B C
end	Pop * then +	[ ]	A B C * +

Result

```
A B C * +
```

Precedence and associativity: two separate rules

Precedence

$A + B \times C$

Multiply first: $A + (B \times C)$.

Associativity

$A - B - C$

Subtract left to right: $(A - B) - C$.

Precedence and associativity: two separate rules

Precedence

$A + B \times C$

Multiply first: $A + (B \times C)$.

Postfix: `A B C * +`

Associativity

$A - B - C$

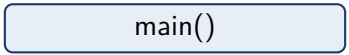
Subtract left to right: $(A - B) - C$.

Postfix: `A B - C -`

Important exception

If exponentiation is defined as **right-associative**, A^{B^C} groups as $A^{(B^C)}$, giving postfix `A B C ^ ^`.

main starts and calls compute

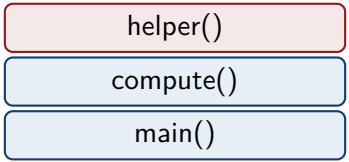


Why LIFO?

The most recently called unfinished function must finish before its caller can resume.

Application 3: the function call stack

Most recent unfinished call: helper



Why LIFO?

The most recently called unfinished function must finish before its caller can resume.

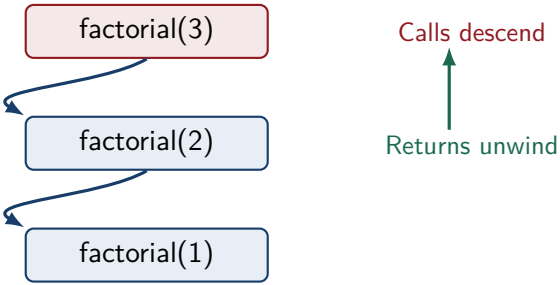
helper returns, then compute returns

main()

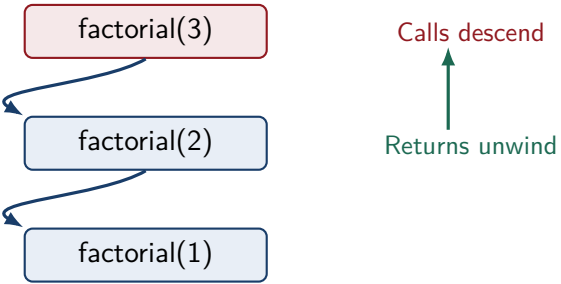
Why LIFO?

The most recently called unfinished function must finish before its caller can resume.

Recursion: each active call has its own frame



Recursion: each active call has its own frame



Return order

factorial(1) = 1, then factorial(2) = 2, then factorial(3) = 6.

Distinguish two ideas

Stack ADT operations are usually $O(1)$; the *number of active recursive calls* determines call-stack depth and memory.

Stacks in daily computing

Undo history
Most recent edit undone first

Browser navigation
A back history can act as a stack

Function execution
Return to most recent caller

Backtracking / DFS
Explore most recently saved choice

Discussion

Can redo be handled with only one undo stack? **Usually a second history or equivalent mechanism is needed to retain undone actions.**

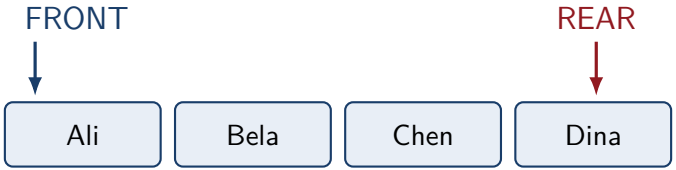
Checkpoint: stack or queue?

Scenario	Appropriate rule
Reverse the most recent operation	**?**
Serve customers by arrival order	**?**
Restore the most recent function caller	**?**
Process jobs in arrival order	**?**

Checkpoint: stack or queue?

Scenario	Appropriate rule
Reverse the most recent operation	**Stack (LIFO)**
Serve customers by arrival order	**Queue (FIFO)**
Restore the most recent function caller	**Stack (LIFO)**
Process jobs in arrival order	**Queue (FIFO)**

Meet the queue: first in, first out



Arrival order: Ali → Bela → Chen → Dina

FIFO rule

Ali arrived first, so Ali must depart first. New people always join at the rear.

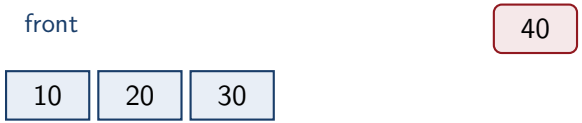
Queue ADT: the operations and their roles

Operation	Meaning	Changes?
enqueue(x)	Insert x at rear	Yes
dequeue()	Remove and return front element	Yes
front()/peek()	Read front without removing it	No
isEmpty()	Report whether the queue is empty	No
size()	Number of waiting elements	No

Think about one element

When a queue holds exactly one item, it is simultaneously **the front** and **the rear**.

Animated queue operation: enqueue 40

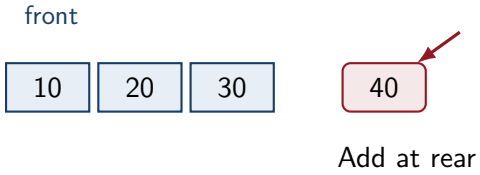


Before enqueue

Ordering invariant

The new element joins at rear. All previously enqueued items remain ahead of it.

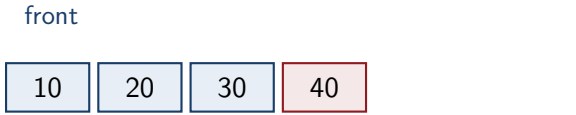
Animated queue operation: enqueue 40



Ordering invariant

The new element joins at rear. All previously enqueued items remain ahead of it.

Animated queue operation: enqueue 40



Queue: 10, 20, 30, 40

Ordering invariant

The new element joins at rear. All previously enqueued items remain ahead of it.

Animated queue operation: dequeue returns 10

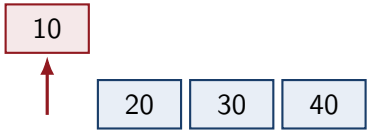


Front is 10

Contrast with stack

Stack removes the *newest*. Queue removes the *oldest*.

Animated queue operation: dequeue returns 10



Return 10

Contrast with stack

Stack removes the *newest*. Queue removes the *oldest*.

Animated queue operation: dequeue returns 10



New front = 20

Contrast with stack

Stack removes the *newest*. Queue removes the *oldest*.

Queue trace: predict front and rear

Operation	Logical queue (front $\rightarrow$ rear)	Returned value
enqueue(5)	[5]	–
enqueue(8)	[5, 8]	–
enqueue(2)	[5, 8, 2]	–
dequeue()	[8, 2]	**5**
enqueue(9)	[8, 2, 9]	–
front()	[8, 2, 9]	**8**

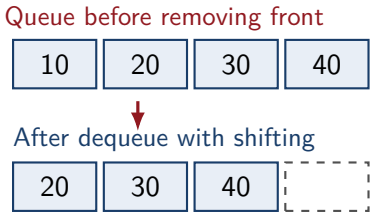
Queue trace: predict front and rear

Operation	Logical queue (front → rear)	Returned value
enqueue(5)	[5]	–
enqueue(8)	[5, 8]	–
enqueue(2)	[5, 8, 2]	–
dequeue()	[8, 2]	**5**
enqueue(9)	[8, 2, 9]	–
front()	[8, 2, 9]	**8**

Explanation

`front()` changes nothing. `dequeue()` must remove the earliest not-yet-removed arrival.

Why a simple array queue wastes work

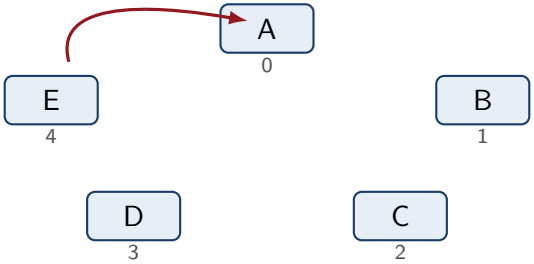


Problem: 3 items shift left
With n items: up to n-1 moves
Cost per removal: $O(n)$

Key idea

A queue can maintain FIFO without moving every remaining item: move the *front index* instead.

Circular queue: reuse the array positions

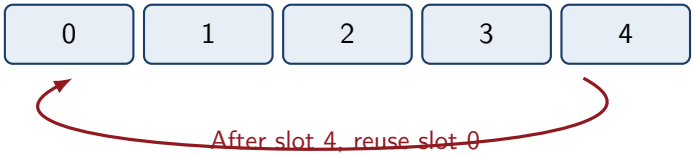


After index 4, continue at index 0

Physical vs logical order

Circular means the *index* wraps, not that items physically travel in a circle. **FIFO order is preserved.**

Modulo is the secret of wraparound



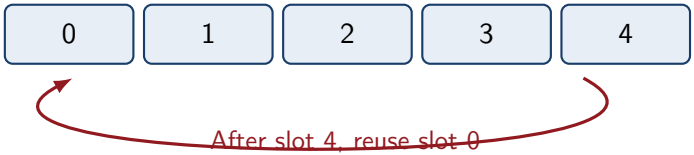
Next index in capacity 5

$$\text{next}(i) = (i + 1) \bmod 5$$

Key idea

Modulo makes the index cycle through valid array positions $0, 1, \ldots, c - 1$.

Modulo is the secret of wraparound



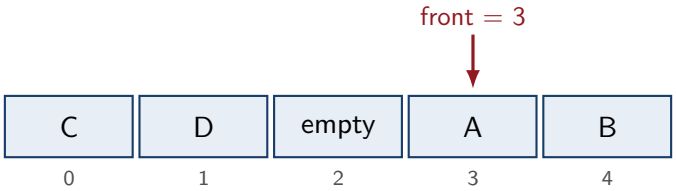
Next index in capacity 5

$$\text{next}(i) = (i + 1) \bmod 5 \qquad \text{next}(3) = 4 \qquad \text{next}(4) = 0$$

Key idea

Modulo makes the index cycle through valid array positions $0, 1, \ldots, c - 1$.

The circular queue needs clear state information



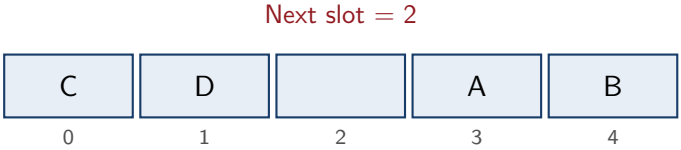
State

Capacity $c = 5$
Front index $f = 3$
Size $n = 4$

Next enqueue slot

$(f + n) \bmod c$
$= (3 + 4) \bmod 5 = \boxed{2}$

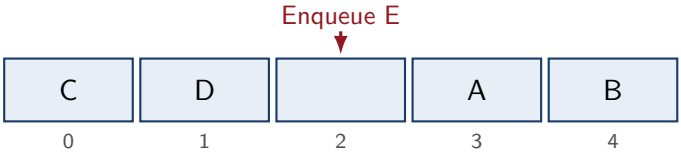
Animated circular enqueue: empty slot 2 is reused



Notice

The front is still at index 3. FIFO sequence is **A, B, C, D, E**, despite physical array positions being C, D, E, A, B.

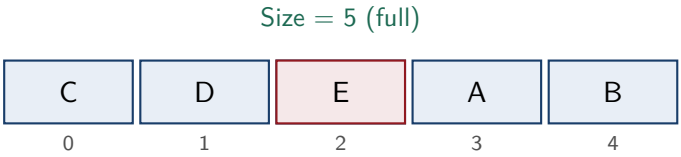
Animated circular enqueue: empty slot 2 is reused



Notice

The front is still at index 3. FIFO sequence is **A, B, C, D, E**, despite physical array positions being C, D, E, A, B.

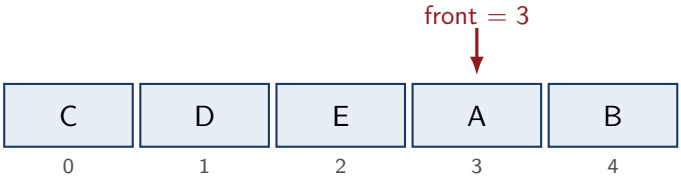
Animated circular enqueue: empty slot 2 is reused



Notice

The front is still at index 3. FIFO sequence is **A, B, C, D, E**, despite physical array positions being C, D, E, A, B.

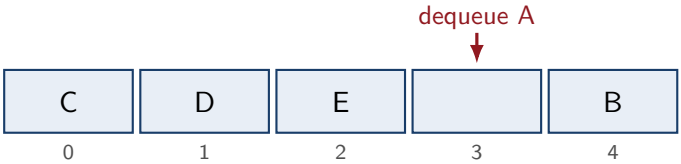
Animated circular dequeue: advance the front



Index rule

New front is $(3 + 1) \bmod 5 = 4$. Size decreases from 5 to 4. No element shifting is required.

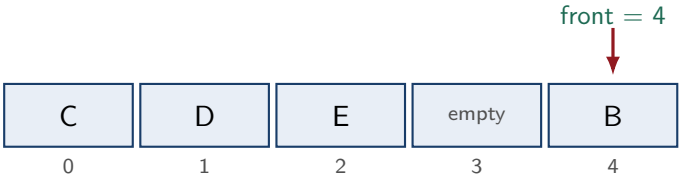
Animated circular dequeue: advance the front



Index rule

New front is $(3 + 1) \bmod 5 = 4$. Size decreases from 5 to 4. No element shifting is required.

Animated circular dequeue: advance the front



Index rule

New front is $(3 + 1) \bmod 5 = 4$. Size decreases from 5 to 4. No element shifting is required.

Circular queue: empty and full are not the same

Empty

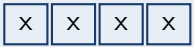
$n = 0$



There is no item to dequeue.

Full

$n = c$

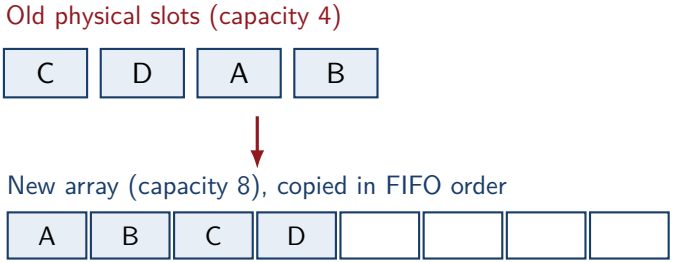


There is no free array slot.

Representation convention matters

Tracking both *front* and *size* allows all c slots to be used. A different design may reserve a slot to distinguish full from empty; do not mix conventions.

Growing a circular queue: copy the logical order



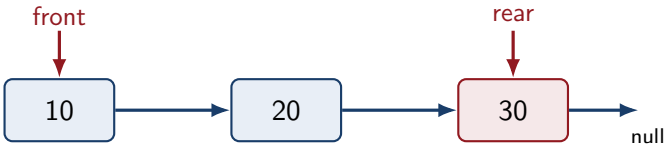
Critical subtlety

If front points to old index 2, raw array order `C D A B` is *not* queue order. Copy `A B C D`, then reset front to 0.

Cost

The resize copies n elements: $O(n)$ for that one enqueue; geometric growth gives amortized $O(1)$ enqueues.

Linked queue: front and rear references



Why two references?

Removing from front and appending at rear both become $O(1)$, without traversing the linked list.

Linked enqueue: connect old rear to the new node



Allocate a node

Special case

If the queue was empty, the new node becomes **both** front and rear.

Linked enqueue: connect old rear to the new node

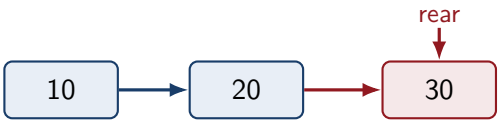


Old rear.next $\rightarrow$ new node

Special case

If the queue was empty, the new node becomes **both** front and rear.

Linked enqueue: connect old rear to the new node

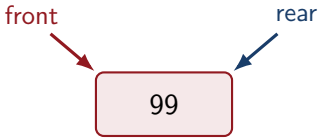


Move rear to new node

Special case

If the queue was empty, the new node becomes **both** front and rear.

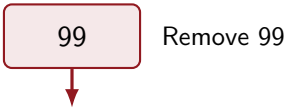
Linked dequeue: the single-element trap



Correctness rule

When the last node is removed, **both** front and rear must be cleared. Otherwise the rear reference is stale.

Linked dequeue: the single-element trap



Correctness rule

When the last node is removed, **both** front and rear must be cleared. Otherwise the rear reference is stale.

Linked dequeue: the single-element trap

$$\text{front} = \text{null} \quad \text{and} \quad \text{rear} = \text{null}$$

Queue is now empty

Correctness rule

When the last node is removed, **both** front and rear must be cleared. Otherwise the rear reference is stale.

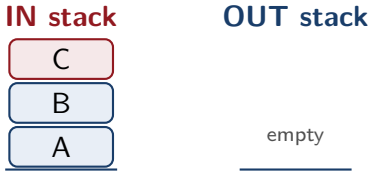
Queue performance: representation matters

Representation	Enqueue	Dequeue	Main trade-off
Shift-based array	$O(1)$ with free slot	$O(n)$	Requires shifts
Fixed circular array	$O(1)$	$O(1)$	Capacity limit
Resizable circular	Amortized $O(1)$	$O(1)$	Occasional resize
Linked queue	$O(1)$	$O(1)$	Node/link overhead

Core lesson

FIFO specifies the behavior. Circular indexing or front/rear references avoid expensive movement of all elements.

Advanced idea: build a queue from two stacks

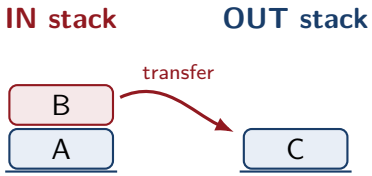


1. Enqueue A, B, C

Push each arrival onto **IN**. The newest item, C, is on top, while A arrived first.

Why the two-stack queue works

Transfer **only when OUT is empty**. Reversing IN puts the oldest element on top of OUT. A transfer may take $O(n)$, but each element transfers at most once, so operations have **amortized** $O(1)$ **time**.

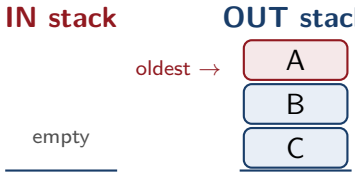


2. Start the transfer

Because **OUT** is empty, pop C from IN and push it onto OUT.

Why the two-stack queue works

Transfer **only when OUT is empty**. Reversing IN puts the oldest element on top of OUT. A transfer may take $O(n)$, but each element transfers at most once, so operations have **amortized** $O(1)$ **time**.



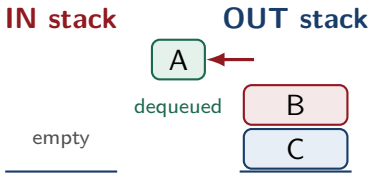
3. Finish the transfer

Next move B, then A. The order reverses: **A is now on top of OUT**.

Why the two-stack queue works

Transfer **only when OUT is empty**. Reversing IN puts the oldest element on top of OUT. A transfer may take $O(n)$, but each element transfers at most once, so operations have **amortized** $O(1)$ **time**.

Advanced idea: build a queue from two stacks



4. Dequeue

Pop from OUT: **A leaves first**. B and C remain, in the correct FIFO order.

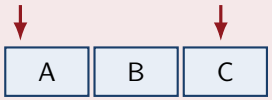
Why the two-stack queue works

Transfer **only when OUT is empty**. Reversing IN puts the oldest element on top of OUT. A transfer may take $O(n)$, but each element transfers at most once, so operations have **amortized** $O(1)$ **time**.

Deque is not the same as a priority queue

Deque: double-ended queue

Insert or remove from **front or rear**.



Priority queue: priority-based

Remove by priority, **not normally FIFO**.

- Urgent first
- Low priority later

Key idea

Priority queues and heap implementations are studied more fully with non-linear structures.

Queues in systems: four familiar examples

Printer jobs
First submitted, first served

Customer requests
Waiting-line order

Buffers
Producer adds; consumer removes

Breadth-first search
Explore neighbors level by level

Important qualification

Real systems can add priorities, scheduling policies, and concurrency constraints; a basic FIFO queue models their simplest ordering behavior.

Choose the ADT from the ordering requirement

Requirement	Best conceptual fit
Undo most recent edit	?
Process registration requests by arrival	?
Allow adding/removing at both ends	?
Handle highest-priority emergency first	?
Check properly nested parentheses	?

Choose the ADT from the ordering requirement

Requirement	Best conceptual fit
Undo most recent edit	**Stack**
Process registration requests by arrival	**Queue**
Allow adding/removing at both ends	**Deque**
Handle highest-priority emergency first	**Priority queue**
Check properly nested parentheses	**Stack**

Decision rule

Identify **which stored element must leave next**.
That usually tells you which ADT is appropriate.

One comparison table to remember

Question	Stack	Queue	Deque
Order	LIFO	FIFO	Two-ended access
Insert at	Top	Rear	Either end
Remove at	Top	Front	Either end
Read next at	Top	Front	Selected end
Example	Undo	Print line	Sliding window

Performance perspective

Efficient representations make the permitted end operations $O(1)$ or amortized $O(1)$.
The total stored elements require $O(n)$ space.

Common conceptual mistakes

1. Confusing peek with pop: inspection is *not* removal.
2. Confusing stack top with queue front: they obey *different ordering rules*.
3. Treating an array's physical order as the logical order of a *wrapped circular queue*.
4. Forgetting the empty-state condition before removal.
5. Leaving a linked queue's rear reference unchanged after removing its last node.
6. Claiming every dynamic-array insertion is worst-case $O(1)$ rather than *amortized* $O(1)$.

Final classroom questions: reason before revealing

1. A stack stores [4, 8, 12] bottom to top. What does `pop()` return?
2. A FIFO queue stores [4, 8, 12] front to rear. What does `dequeue()` return?
3. With capacity 6 and front index 5, what is the next index?
4. Why does `([)]` fail bracket matching?
5. Why can a linked queue remove from front in constant time?

Final classroom questions: reason before revealing

1. A stack stores [4, 8, 12] bottom to top. What does `pop()` return? **12**
2. A FIFO queue stores [4, 8, 12] front to rear. What does `dequeue()` return? **4**
3. With capacity 6 and front index 5, what is the next index? **0**
4. Why does `([)]` fail bracket matching? **A closing ')' sees '[' on top.**
5. Why can a linked queue remove from front in constant time? **It updates the front reference directly.**

Takeaways and preparation for the next lecture

What to remember

- **Stack = LIFO**; queue = FIFO.
- ADT rules are independent of memory representation.
- Arrays, circular arrays, and linked nodes implement these rules.

Where it leads

- Balanced expressions and function calls use stacks.
- Buffers, waiting lines, and BFS use queues.
- Trees, heaps, and graphs generalize structured data access.

Independent practice

Draw the queue's physical slots and logical order after a wraparound; trace a bracket string; explain one $O(1)$ vs $O(n)$ design choice.

Suggested reference and further reading

Primary text

Michael H. Goldwasser, Michael T. Goodrich, and Roberto Tamassia, *Data Structures and Algorithms in Python*, Wiley.
Relevant reading: stack, queue, linked representations, dynamic arrays, and fundamental data-structure operations.

What to practise next

Draw operations using pencil and paper first. Then compare your sequence with the ADT's rules; code is secondary to understanding the invariants.